

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

PENGANTAR MIKROKONTROLER

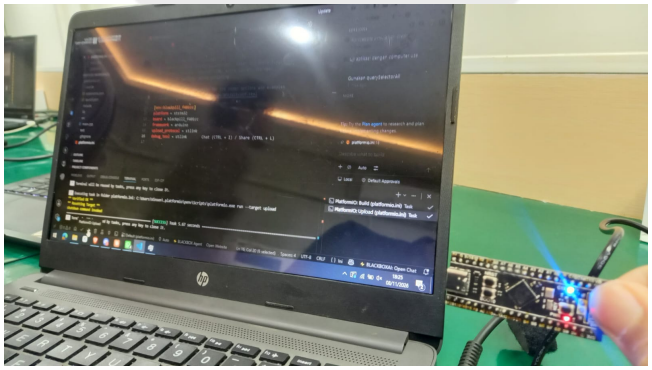
Anisa Hasna Mufida - 5024241071

2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Membuat project Modul 1 pada Visual Studio Code menggunakan PlatformIO sesuai konfigurasi yang telah ditentukan.
2. Mengatur file platformio.ini dengan memilih platform STM32, board $blackpill_f401cc$, framework Arduino, serta ST-Link sebagai metode upload dan debugging.
3. Menghubungkan ST-Link dengan board STM32 Blackpill menggunakan kabel sesuai pasangan pin SWDIO, SWCLK, GND, dan 3.3V.
4. Menghubungkan ST-Link ke komputer melalui port USB dan memastikan koneksi dengan board telah terpasang dengan benar.
5. Menuliskan kode program Blink pada file src/main.cpp dengan menggunakan pin PC13 sebagai output LED onboard.
6. Melakukan proses Build untuk mengompilasi program dan memastikan tidak terdapat kesalahan pada kode.
7. Melakukan proses Upload untuk mengunggah program ke mikrokontroler STM32 Blackpill melalui ST-Link.
8. Mengamati kondisi LED onboard pada pin PC13 untuk memastikan LED menyala selama 500 milidetik dan mati selama 500 milidetik secara berulang.

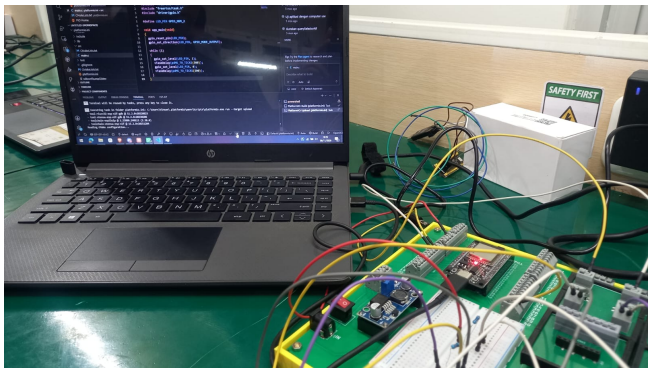


Gambar 1: Dokumentasi Hasil Percobaan 1

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Mengubah konfigurasi file platformio.ini dengan memilih platform ESP32, board esp32dev, dan framework ESP-IDF.
2. Menunggu hingga proses refresh konfigurasi PlatformIO selesai agar project menggunakan konfigurasi terbaru.
3. Menghubungkan board ESP32 ke komputer menggunakan kabel USB dan memastikan komputer telah mengenali port perangkat.

4. Menghapus file src/main.cpp, kemudian membuat file baru bernama src/main.c sebagai tempat penulisan program.
5. Menuliskan kode program Blink menggunakan framework ESP-IDF dengan memanfaatkan fungsi GPIO dan FreeRTOS untuk mengatur kondisi LED onboard pada pin GPIO 2.
6. Melakukan proses Build untuk mengompilasi program dan memastikan tidak terdapat kesalahan.
7. Melakukan proses Upload untuk mengunggah program ke board ESP32.
8. Mengamati kondisi LED onboard untuk memastikan LED dapat menyala dan mati secara bergantian dengan interval 500 milidetik.
9. Menganalisis perbedaan struktur program dan metode pengendalian GPIO antara framework ESP-IDF dan Arduino.



Gambar 2: Dokumentasi Rangkaian dan Hasil Percobaan 2

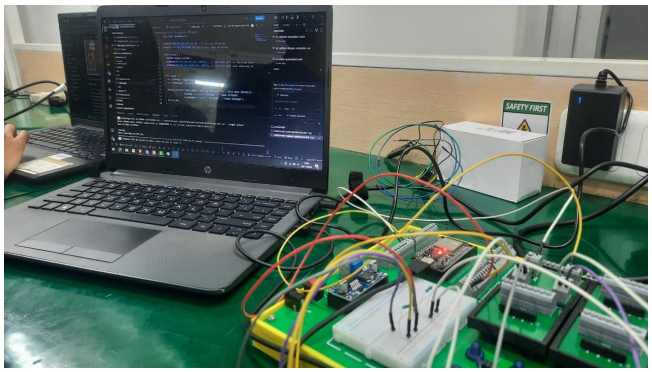
1.3 Percobaan 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

1. Mengatur konfigurasi file platformio.ini menggunakan platform ESP32, board esp32dev, dan framework Arduino.
2. Menghapus file src/main.c, kemudian membuat kembali file src/main.cpp sebagai tempat penulisan program.
3. Merangkai dua pushbutton pada GPIO 32 dan GPIO 33 ESP32 dengan menggunakan resistor pull-up dan pull-down eksternal sebesar 10 k sesuai skema rangkaian pada modul.
4. Menuliskan Kode Program A pada file src/main.cpp untuk membaca kondisi kedua tombol menggunakan konfigurasi input biasa (INPUT).
5. Melakukan proses Build dan Upload, kemudian membuka Serial Monitor dengan baud rate 115200 untuk mengamati perubahan kondisi tombol ketika ditekan maupun tidak ditekan.
6. Melepaskan kedua resistor eksternal dari rangkaian, sementara kedua pushbutton tetap terhubung pada pin GPIO yang sama sesuai skema pull-up dan pull-down internal.

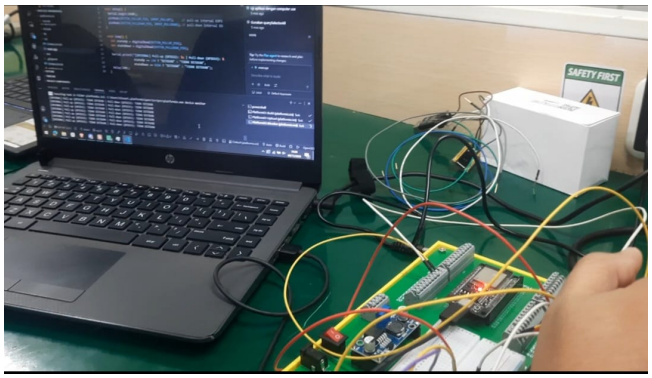
7. Mengganti program pada src/main.cpp menggunakan Kode Program B dengan mengaktifkan konfigurasi $INPUT_PULLUP$ pada GPIO 32 dan $INPUT_PULLDOWN$ pada GPIO 33.

8. Melakukan proses Build dan Upload kembali, kemudian menguji kedua tombol dan mengamati hasil pembacaannya melalui Serial Monitor.

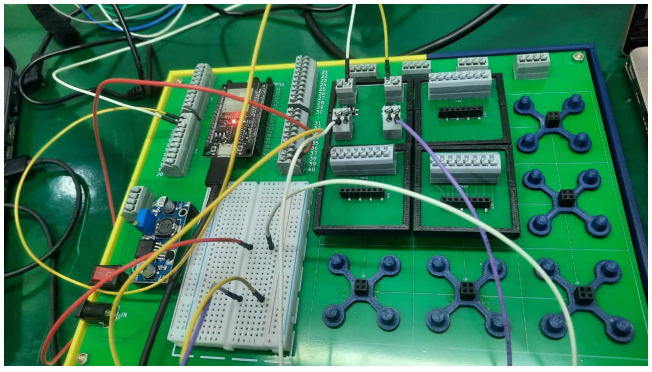
9. Membandingkan hasil pembacaan tombol pada penggunaan resistor eksternal dan internal, serta mengamati perbedaan kondisi logika HIGH dan LOW ketika tombol ditekan maupun tidak ditekan.



Gambar 3: Dokumentasi Hasil dan Rangkaian Bagian A



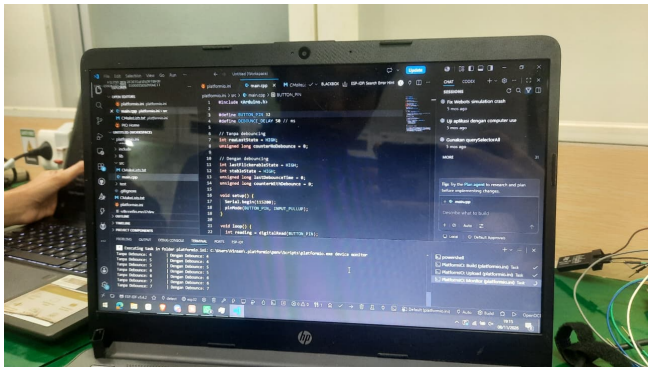
Gambar 4: Dokumentasi Hasil Bagian B



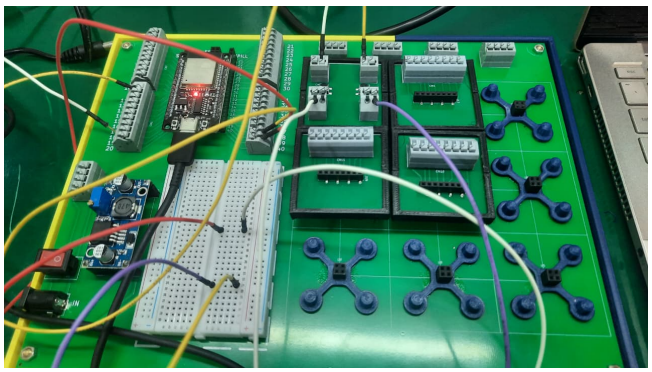
Gambar 5: Dokumentasi Rangkaian Bagian B

1.4 Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

1. Mengatur konfigurasi project menggunakan platform ESP32, board esp32dev, dan framework Arduino.
2. Merangkai satu pushbutton pada GPIO 25 ESP32 dengan menghubungkan salah satu kaki tombol ke pin GPIO dan kaki lainnya ke GND menggunakan konfigurasi pull-up internal.
3. Menuliskan program debouncing pada file src/main.cpp dengan menggunakan dua penghitung, yaitu counterNoDebounce dan counterWithDebounce, untuk membandingkan pembacaan tombol tanpa dan dengan debouncing.
4. Melakukan proses Build dan Upload, kemudian membuka Serial Monitor untuk mengamati hasil pembacaan kedua penghitung.
5. Menekan tombol satu kali dan membandingkan perubahan nilai kedua penghitung untuk mengetahui pengaruh bouncing pada tombol mekanik.
6. Mengulangi pengujian dengan variasi kecepatan penekanan tombol, baik secara cepat maupun lambat, kemudian mengamati perbedaan hasil penghitungannya.
7. Mengubah nilai $DEBOUNCE_DELAY$ dari 50 milidetik menjadi 10 milidetik, kemudian melakukan Build dan Upload kembali untuk mengamati hasil pembacaan tombol.
8. Mengubah nilai $DEBOUNCE_DELAY$ menjadi 200 milidetik, kemudian melakukan Build dan Upload kembali serta menguji tombol dengan menekannya secara cepat dan berulang.
9. Membandingkan hasil pengujian menggunakan nilai debounce 10, 50, dan 200 milidetik untuk mengetahui pengaruh durasi debouncing terhadap kestabilan pembacaan dan kecepatan respons tombol.



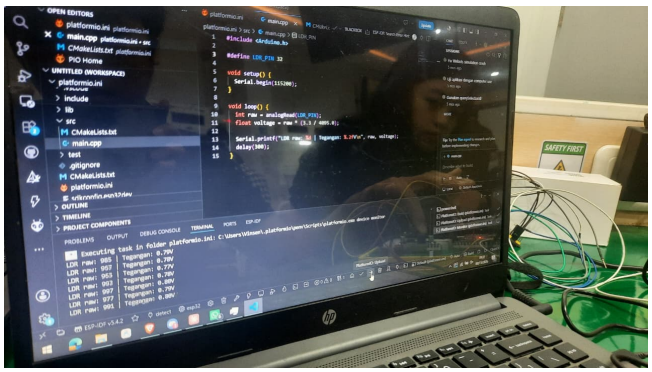
Gambar 6: Dokumentasi Hasil Percobaan 4



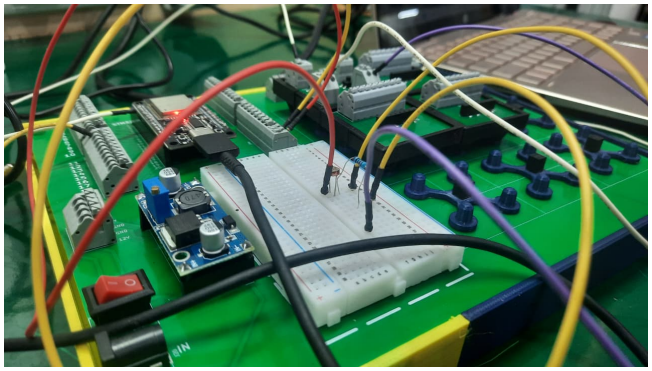
Gambar 7: Dokumentasi Rangkaian Percobaan 4

1.5 Percobaan 5: Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

1. Mengatur konfigurasi project menggunakan platform ESP32, board esp32dev, dan framework Arduino.
2. Merangkai komponen LDR dan resistor sebesar $10k$ sebagai pembagi tegangan sesuai skema rangkaian pada modul.
3. Menghubungkan salah satu kaki LDR ke sumber tegangan 3,3V, sedangkan kaki lainnya dihubungkan ke GPIO 34 dan resistor $10k$ yang terhubung ke GND.
4. Menuliskan program pembacaan ADC pada file src/main.cpp untuk membaca nilai analog dari LDR dan mengonversinya menjadi nilai tegangan.
5. Melakukan proses Build dan Upload untuk mengunggah program ke board ESP32.
6. Membuka Serial Monitor untuk mengamati nilai ADC mentah dan hasil konversi tegangan yang ditampilkan secara berkala.
7. Menutup permukaan LDR menggunakan tangan untuk mengamati perubahan nilai ADC dan tegangan yang terbaca.
8. Memberikan pencahayaan pada LDR menggunakan lampu, kemudian mengamati perubahan nilai ADC dan tegangan yang ditampilkan pada Serial Monitor.
9. Membandingkan hasil pembacaan pada kondisi pencahayaan yang berbeda untuk memahami hubungan antara intensitas cahaya, perubahan tegangan, dan nilai ADC yang dihasilkan.



Gambar 8: Dokumentasi Hasil Percobaan 5



Gambar 9: Dokumentasi Rangkaian Percobaan 5

2 Analisis Hasil Percobaan

Pada Percobaan 1, program Blink berhasil diunggah ke STM32 Blackpill melalui ST-Link V2 dengan jalur SWD, dan LED onboard pada pin PC13 berkedip dengan selang 500 ms. Hasil ini sesuai dengan teori bahwa LED onboard Blackpill bersifat aktif LOW, sehingga LED menyala ketika pin diberi logika LOW dan padam ketika diberi logika HIGH. Keberhasilan proses unggah bergantung pada sambungan SWDIO, SWCLK, GND, dan 3V3 antara ST-Link dan board, serta pengaturan platformio.ini yang tepat, yaitu platform ststm32, board blackpill$_f401cc, danupload_protocolstlink.Kabelyanglonggaratau driverST-Linkyangbelumterpasangdapatmenyebabkanboardtidakterdeteksi.$

Pada Percobaan 2, program Blink dengan framework ESP-IDF berhasil dijalankan pada ESP32 dan LED pada GPIO 2 berkedip sesuai selang yang diprogram. Dibandingkan dengan Arduino, ESP-IDF menuntut struktur program yang lebih eksplisit, yaitu titik masuk app$_main(), konfigurasipinmelaluigpio_reset_p$
$IDFadalahframeworknativedariEspressifdengantingkatabstraksilebihrendahdankontrollebihbesarterhadapp$

Pada Percobaan 3, tombol dengan pull-up eksternal terbaca HIGH saat tidak ditekan dan LOW saat ditekan, sedangkan tombol dengan pull-down eksternal terbaca LOW saat tidak ditekan dan HIGH saat ditekan. Setelah resistor eksternal dilepas dan diganti dengan $INPUT_PULLUP$ dan $INPUT_PULLDOWN$, perilaku pembacaan tetap sama. Hal ini sesuai dengan teori bahwa resistor internal ESP32 berfungsi seperti resistor eksternal, yaitu menarik pin ke level tegangan yang pasti ketika tombol tidak ditekan sehingga pin tidak berada pada kondisi floating. Pin yang floating dapat membaca nilai acak karena menangkap noise dari lingkungan. Hasil yang tidak sesuai dapat terjadi apabila resistor eksternal belum dilepas sepenuhnya, karena dua penarik tegangan yang berlawanan bekerja pada satu pin, atau apabila kaki tombol dipasang pada pasangan kaki yang salah di brea-

dboard.

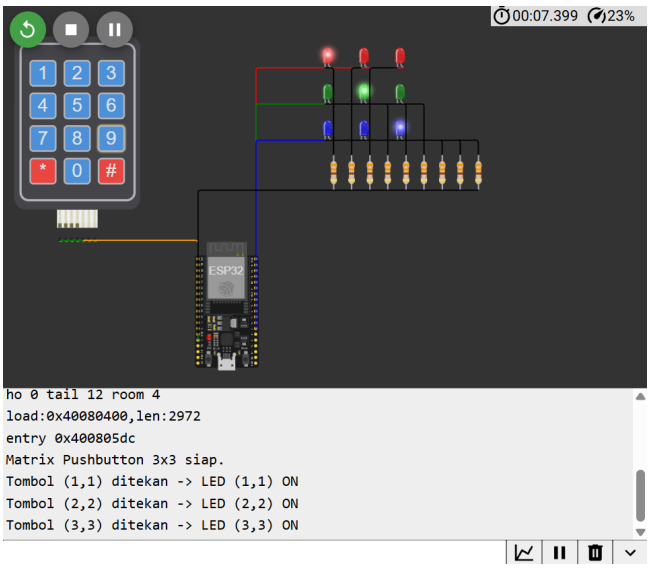
Pada Percobaan 4, penghitung tanpa debounce menghasilkan hitungan yang lebih banyak daripada jumlah penekanan tombol sebenarnya, sedangkan penghitung dengan debounce menghasilkan hitungan yang sama dengan jumlah penekanan. Selisih ini muncul karena kontak mekanik pada tombol tidak menutup dengan mulus, sehingga sinyal memantul beberapa kali antara HIGH dan LOW dalam waktu sangat singkat. Program tanpa debounce membaca setiap pantulan tersebut sebagai penekanan baru, sedangkan program dengan debounce hanya menerima perubahan yang stabil selama $DEBOUNCE_DELAY$. Pada nilai 10 ms, bouncing masih dapat lolos karena waktu tunggu yang terlalu singkat, sehingga hitungan berlebih masih mungkin muncul. Pada nilai 50 ms, bouncing tersaring dengan baik dan respons tombol tetap terasa cepat. Pada nilai 200 ms, bouncing tersaring sepenuhnya tetapi respons menjadi lambat dan penekanan yang cepat dapat tidak terbaca. Hasil ini sesuai dengan teori bahwa waktu debounce yang ideal berada pada kisaran 20 sampai 50 ms.

Pada Percobaan 5, nilai ADC yang dibaca dari GPIO 34 berubah mengikuti intensitas cahaya yang mengenai LDR. Pada rangkaian pembagi tegangan dengan LDR di sisi 3,3 V dan resistor 10 k di sisi GND, hambatan LDR turun ketika cahaya meningkat, sehingga tegangan pada GPIO 34 naik dan nilai ADC membesar. Sebaliknya, ketika LDR ditutup, hambatannya naik dan nilai ADC mengecil. Nilai ADC berada pada rentang 0 sampai 4095 sesuai resolusi 12 bit pada ESP32, dan tegangan dapat dihitung dengan persamaan V = nilai ADC × 3,3 / 4095. Hasil ini sesuai dengan teori bahwa ADC dapat merepresentasikan banyak kondisi, berbeda dengan input digital yang hanya membedakan HIGH dan LOW. Nilai yang terbaca dapat dipengaruhi oleh toleransi resistor, perubahan cahaya sekitar, noise pada pembacaan, serta ketidaklinearan ADC ESP32 pada ujung rentang tegangan, sehingga nilai tidak selalu tepat 0 atau 4095.

Secara keseluruhan, seluruh percobaan berhasil dan hasilnya sesuai dengan teori yang dipelajari. Faktor yang paling memengaruhi keberhasilan adalah ketepatan pemasangan kabel dan komponen, serta pengaturan platformio.ini. Percobaan ini menunjukkan bahwa pembacaan input digital yang andal membutuhkan pull-up atau pull-down agar pin tidak floating dan debouncing agar pantulan sinyal tidak terbaca sebagai beberapa penekanan, sedangkan ADC memungkinkan pembacaan besaran fisik yang bernilai kontinu.

3 Hasil Tugas Modul

Berikut merupakan dokumentasi hasil simulasi Matrix Pushbutton 3x3 mengendalikan Matrix LED 3x3.



Gambar 10: Dokumentasi simulasi Matrix Pushbutton 3x3 mengendalikan Matrix LED 3x3

Berikut merupakan jawaban pertanyaan analisis yang diberikan dalam modul praktikum.

1. Debouncing tetap diperlukan pada rangkaian matrix pushbutton karena setiap tombol mekanik memiliki kontak fisik yang dapat mengalami bouncing ketika ditekan maupun dilepaskan. Bouncing menyebabkan perubahan sinyal HIGH dan LOW secara berulang dalam waktu singkat sebelum akhirnya mencapai kondisi stabil. Meskipun hanya satu tombol yang ditekan pada satu waktu, perubahan sinyal tersebut tetap dapat terbaca sebagai beberapa kali penekanan oleh mikrokontroler. Akibatnya, LED dapat mengalami perubahan status yang tidak sesuai dengan kondisi tombol sebenarnya. Oleh karena itu, diperlukan debouncing berbasis software untuk memastikan bahwa perubahan kondisi tombol telah stabil selama waktu tertentu sebelum diproses. Pada rancangan ini digunakan waktu debounce sebesar 50 milidetik sehingga LED hanya merespons perubahan tombol yang telah dinyatakan stabil.

2. Pada rangkaian ini digunakan konfigurasi pull-up internal ESP32 melalui fungsi $INPUT_PULLUP$ pada setiap pin kolom tombol. Dengan konfigurasi tersebut, pin kolom akan berada pada kondisi HIGH ketika tombol tidak ditekan karena mendapatkan referensi tegangan melalui resistor pull-up internal. Ketika tombol ditekan pada baris yang sedang diaktifkan dengan kondisi LOW, pin kolom akan terhubung ke GND sehingga menghasilkan kondisi LOW. Kondisi pin tidak boleh dibiarkan floating karena pin input tidak memiliki referensi logika yang pasti sehingga dapat membaca perubahan HIGH dan LOW secara tidak menentu akibat gangguan listrik di sekitarnya. Hal tersebut dapat menyebabkan mikrokontroler mendeteksi penekanan tombol yang sebenarnya tidak terjadi. Oleh karena itu, penggunaan pull-up internal diperlukan untuk menjaga kestabilan pembacaan input digital.

3. Perbedaan mendasar antara digital input dan analog input terletak pada jenis sinyal yang dibaca oleh mikrokontroler. Digital input hanya mengenali dua kondisi logika, yaitu HIGH dan LOW, sehingga cocok digunakan untuk membaca kondisi tombol yang ditekan atau tidak ditekan. Sementara itu, analog input menggunakan ADC (Analog to Digital Converter) untuk mengubah tegangan analog menjadi nilai digital yang dapat diproses oleh mikrokontroler. Pada ESP32, ADC memiliki resolusi 12-bit sehingga mampu menghasilkan nilai pembacaan dari 0 hingga

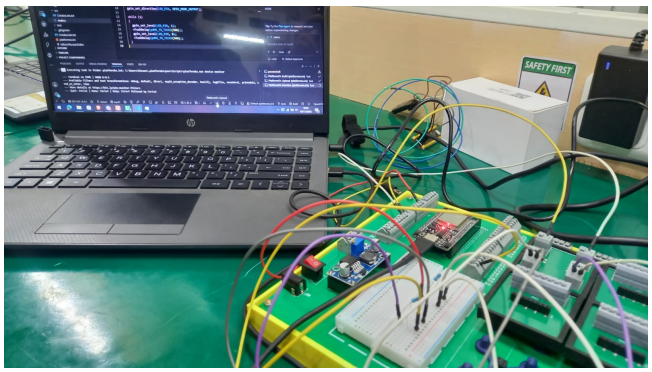
4095. Dengan demikian, ADC dapat merepresentasikan banyak tingkat tegangan yang berbeda, bukan hanya dua kondisi seperti pada digital input. Apabila tombol digantikan dengan LDR, perubahan intensitas cahaya akan memengaruhi resistansi LDR dan menghasilkan perubahan tegangan pada rangkaian pembagi tegangan. Perubahan tegangan tersebut kemudian dibaca oleh ADC sehingga mikrokontroler dapat membedakan berbagai tingkat intensitas cahaya berdasarkan nilai ADC yang diperoleh.

4 Kesimpulan

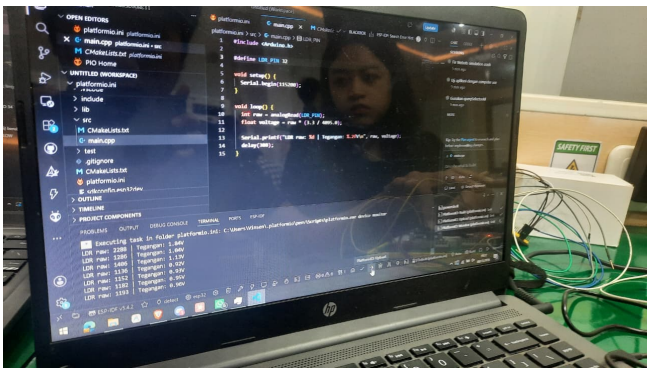
Berdasarkan praktikum Pengantar Mikrokontroler yang telah dilakukan, seluruh percobaan berhasil dan hasilnya sesuai dengan teori. Pemrograman Blink pada STM32 Blackpill dengan framework Arduino dan pada ESP32 dengan framework ESP-IDF menunjukkan bahwa satu fungsi yang sama dapat dibuat dengan tingkat abstraksi yang berbeda, di mana Arduino lebih sederhana sedangkan ESP-IDF memberi kontrol lebih besar melalui FreeRTOS. Pembacaan tombol memerlukan resistor pull-up atau pull-down, baik eksternal maupun internal, agar pin tidak berada pada kondisi floating dan menghasilkan pembacaan yang pasti. Tombol mekanik juga menghasilkan bouncing yang dapat terbaca sebagai beberapa penekanan, sehingga debouncing dengan waktu tunggu sekitar 20 sampai 50 ms diperlukan agar pembacaan stabil tanpa membuat respons terlalu lambat. Selain itu, pembacaan ADC pada GPIO analog dengan LDR menunjukkan bahwa satu pin dapat merepresentasikan nilai 0 sampai 4095 sesuai intensitas cahaya, sehingga mampu membaca besaran yang kontinu, tidak hanya kondisi HIGH dan LOW seperti pada input digital.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 11: Dokumentasi alat dan bahan yang digunakan saat melakukan praktikum



Gambar 12: Dokumentasi saat melakukan praktikum